

Parte 4. Conceptos

Responde esta parte en el documento `parte_4/parte_4.pdf`. Incluye cada pregunta y su respuesta.

1. Explica la diferencia entre Git y GitHub.

- Git es un sistema de control de versiones distribuido de código abierto que se ejecuta de manera local en tu computadora. Su función principal es registrar, rastrear, comparar y gestionar cada uno de los cambios realizados en el código fuente de un proyecto a lo largo del tiempo, permitiendo trabajar con ramas, deshacer errores y fusionar trabajos de forma offline.
- GitHub es una plataforma web en la nube (un servicio de alojamiento) construida alrededor de Git. Funciona como un centro colaborativo donde los desarrolladores pueden almacenar copias remotas de sus repositorios, facilitando la revisión de código, la gestión de incidentes, la automatización y la colaboración en equipo mediante herramientas como Pull Requests. En resumen, Git es el motor local de control de versiones y GitHub es el servidor web remoto que aloja y socializa los proyectos.

2. Explica para qué sirve `.gitignore`.

El archivo `.gitignore` es un archivo de configuración de texto plano que se coloca en la raíz del proyecto para indicarle a Git qué archivos o carpetas debe omitir por completo y no rastrear. Su propósito es evitar que se añadan al control de versiones archivos temporales del sistema operativo, credenciales de acceso privadas, llaves API, bases de datos locales, entornos virtuales o cachés de compilación. Su uso garantiza que el repositorio se mantenga limpio, seguro, ligero y libre de archivos basura que son específicos del equipo de desarrollo local de cada usuario.

3. Explica por qué `.venv` no debe almacenarse normalmente en GitHub.

La carpeta `.venv` contiene el entorno virtual de Python, el cual almacena los ejecutables del intérprete y copias o enlaces de todas las bibliotecas instaladas (las cuales pueden sumar cientos de megabytes o incluir rutas absolutas del sistema operativo del equipo donde se creó). No debe subirse a GitHub por tres razones principales:

1. Peso y redundancia innecesaria: Multiplicaría masivamente el tamaño del repositorio en la nube con archivos binarios y dependencias pesadas que se pueden reinstalar fácilmente.
2. Incompatibilidad de sistemas: Las rutas y archivos compilados dentro de un entorno virtual suelen estar atados a la arquitectura específica del sistema operativo local

(Windows, macOS o Linux) de quien lo generó, por lo que no funcionarán directamente en otra computadora.

3. Buenas prácticas: En lugar de subir el entorno virtual, se comparte un archivo ligero de texto llamado `requirements.txt`, permitiendo que cada colaborador cree su propio entorno virtual limpio de manera local.

4. Explica para qué sirve `requirements.txt`.

El archivo `requirements.txt` es un archivo de texto estándar utilizado en proyectos de Python para listar de forma explícita todas las bibliotecas externas y dependencias (junto con sus versiones específicas) que la aplicación necesita para ejecutarse correctamente. Su propósito fundamental es asegurar la reproducibilidad del entorno de desarrollo. Cualquier desarrollador que clone el repositorio puede instalar de manera automática y exacta las mismas herramientas y versiones ejecutando un único comando (`pip install -r requirements.txt`), evitando conflictos de compatibilidad de código.

5. Explica la diferencia entre Stage, Commit y Push.

Representan tres momentos sucesivos y distintos dentro del flujo de trabajo de Git:

- Stage (Área de Preparación / Staging Area): Es un espacio de almacenamiento intermedio. Cuando modificas archivos, se utiliza `git add` para enviarlos al Stage; esto significa que estás seleccionando y agrupando conscientemente qué cambios específicos deseas incluir en tu próxima instantánea.
- Commit: Es el momento en el que el contenido que estaba en el Stage se registra de forma permanente y con un identificador único (hash) en el historial local de Git. Cada commit es una versión estable y documentada del proyecto acompañada de un mensaje descriptivo.
- Push: Es la acción de comunicación con el servidor remoto. Utilizando `git push`, tomas los commits que se han acumulado y guardado en tu historial local y los envías a la nube (por ejemplo, a GitHub) para actualizar el repositorio remoto.

6. Explica por qué un repositorio puede tener varios commits antes de realizar un push.

Un repositorio puede (y suele) acumular múltiples commits antes de hacer un push debido a que el flujo de trabajo de Git está diseñado para ser iterativo y trabajar offline (sin necesidad de conexión constante a internet).

Hacer varios commits locales permite al desarrollador:

1. Registrar avances atómicos: Dividir el desarrollo en pequeñas unidades lógicas de trabajo (por ejemplo, un commit para la estructura base, otro para corregir un error y otro para añadir una función).

2. Mantener un historial limpio y comprensible: Facilitar la depuración y el rastreo de errores mediante mensajes claros asociados a cada etapa pequeña del desarrollo.
3. Trabajar de forma independiente: Acumular avances de forma segura localmente antes de compartirlos de golpe con el equipo remoto mediante un único push sincronizado.